



语言分析技术实验报告

实验 4：文本规范化

小组成员：田佳铭（22920172204211）

迟宇希（22920172204083）

颜思琦（22920172204255）

院 系：信息学院人工智能系

专 业：智能科学与技术

年 级：2017 级

指导教师：史晓东

2020 年 6 月 20 日

目录

一、实验目的.....	2
二、实验环境及工具.....	2
三、附件与参考资料等说明.....	2
四、整体设计思想.....	3
4.1 繁体字.....	4
4.2 字母符号.....	4
4.3 拆字型.....	4
4.4 基于分类的选择策略.....	4
五、具体实现思路.....	5
5.1 繁体转简体.....	5
5.2 字母符号转换.....	5
5.2.1 字词拼音映射.....	5
5.2.2 拼音/单词识别.....	6
5.2.3 拼音转汉字.....	8
5.3 拆字倒映射.....	9
5.4 分类与选择策略.....	9
5.5 匹配策略.....	10
5.6 命令行输入实现.....	11
六、测试.....	12
6.1 测试结果.....	12
6.1.1 字符转换结果.....	12
6.1.2 拆字型转换结果.....	12
6.1.3 测试集识别结果.....	13
6.2 结果说明.....	13
七、遇到的问题与解决.....	15
八、感想.....	16

一、实验目的

目前微信、微博上的语言经常很不规范。写个程序，把这些句子中的非规范词，恢复成正常的词。

文本可以用文件表示，仅测试一个句子的话，用-s 选项表示。

二、实验环境及工具

因为是组队完成，大家使用的 IDE 有所不同，例如田佳铭使用的是 IDLE，迟宇希使用的是 pycharm 等，但大家的编程语言都是 Python3，实践证明，代码是可以在这两个平台中运行的。

为了保证可移植性，主要代码都放在了同一个目录下。

三、附件与参考资料等说明

本次实验小组分工完成完成，参考了不少的开源资料。

资料查找与整合时间：两周左右（抽出课余时间完成），实验代码所花时间约 3 天（整合成一天 24 小时的结果）。

小组分工：

（1）田佳铭：提供部分代码实现整体思路。负责繁体转简体、获取拆字倒映射、训练 Word2Vec 模型部分的代码实现，并负责最后的代码整合、文件整理和测试等工作。

（2）迟宇希：负责拼音汉字倒映射、拼音常用词倒映射、拼音分词并转汉字部分的代码实现，并负责主程序流程代码编写。

（3）颜思琦：参与字母符号转换，资料收集，数据库查找，英文语料库建立等工作。

附件中文件说明（更详细的说明见文件中的各个文件夹中的 readme.txt）：

- (1) i_chazi.txt: 拆字倒映射模型文件
- (2) model_train.pkl: Word2Vec 模型
- (3) pinyin_hanzi.pkl: 拼音-汉字常用词映射
- (4) pinyinLib.txt: 本地拼音库
- (5) 规范词典 2000 条.txt: 实验本身就有的规范词典
- (6) langconv.py、zh_wiki.py: 和繁体转简体有关的中间代码
- (7) testdata.txt: 一个测试文件
- (8) 规范化测试集 1000 条.txt: 实验本身的测试集
- (9) client.py: 主要程序文件，实现实验要求的文件
- (10) test.py: 主要程序文件，测试识别率的文件
- (11) get_model 文件夹: 获取各种模型的文件夹

本实验所使用到的开源训练集、开源库有（只列举主要的一些）：

- (1) jieba: 用于分词的 python 库
- (2) gensim: 用以训练 Word2Vec 模型的 python 库
- (3) pickle 和 json: 用于保存和读取模型文件的 python 库
- (4) chaizi-jt.txt: 来自 github 开源项目的简体拆字字典
- (5) langconv.py、zh_wiki.py: github 上开源项目 nstools 中的两个文件，与简体转繁体有关
- (6) ./get_model/get_W2V_model/train: 训练 Word2Vec 模型的训练集，来源于复旦大学的一个开源中文文本分类语料的简体部分，包括很多子类的训练资料，例如艺术类、计算机类等（文件中只放入了几个样例，真实的训练样本为 80 多 M）。
- (7) Pinyin2Hanzi、pypinyin: 与拼音有关的 python 库

四、整体设计思想

因为要考虑到所有的不规范的情况很困难，也不太现实，我们小组主要针对于其中的少数不规范的情况。

4.1 繁体字

第一种情况主要是繁体字的情况，因为网络微博体文字中，许多年轻人喜爱用繁体字来表达自己的心情，这样的不规范词是我们需要规范修正的一类，我们将其全部规范成简体字，这也方便后续的一些处理。

4.2 字母符号

第二种情况是带有字母符号的词，这是我们处理中需要考虑比较多的部分。带有字母符号的词主要分为三种：拼音、英语和网络用语简写。

对于英语，我们根据老师要求和测试集示例，对其予以保留。

对于网络用语简写部分，我们将其加入字词-拼音映射部分，使用词向量模型统一判断是否需要替换。

对于拼音部分，我们首先制作了常用字常用词到拼音的倒映射，这样可以快速准确的讲常用的拼音转化成汉字，对于不常用的拼音，我们首先要区分它是否为英语单词，最初我们使用正向最大匹配模式来区分英语单词与拼音，但是通过测试我们发现，这种方法并不是十分可靠，例如“open”就可以匹配成“o, pen”，并被判断为拼音。为了解决这种情况，我们使用了新的英语二元语言模型来进行优化，测试效果优于第一种正向最大匹配的方法。

4.3 拆字型

第三种需要考虑的情况是拆字型，拆字型也是经常出现的一种情况，我们首先训练了常见字的一些拆字映射，然后对测试语句进行匹配，如果匹配成功，则将原文和拆字重组后的字同时加入候选集，并用词向量模型对候选集进行选择，判断是否需要替换。增加运用词向量模型进行判断的原因是，类似于“日月”这种词，既是常用字的拆字型，又是一个常见词组，这是我们需要通过上下文才能判断是否需要替换。

4.4 基于分类的选择策略

综上，对于最主要的三类不规范的情况，我们提出了利用词向量模型基于分类的选择策略，通过上下文关系来选择此处可替换的最佳规范词。这种方法假设

为句子是前后文呼应的，事实上测试结果也表明大多数句子都是前后文有关联的，基于分类的方法比较适用，对于个别前后文毫无关联的句子，替换结果还有待提升。

五、具体实现思路

5.1 繁体转简体

繁体转简体这里主要使用开源项目。这里使用了 github 上开源项目 `nstools` 中的两个文件 `langconv.py`、`zh_wiki.py` 来实现繁体转简体。

5.2 字母符号转换

不规范文本中出现的字母符号有两种可能：英语单词或汉语拼音，我们采取的主要策略是先采用 `n-gram` 判断属于单词还是拼音，单词保留，拼音经过分词处理，使用 `HMM` 转换为相应的汉字。

5.2.1 字词拼音映射

常用字的选取采用了 `GB2312` 编码，`GB2312` 是 `ANSI` 编码里的一种，是一个简体中文字符集，由 6763 个常用汉字和 682 个全角的非汉字字符组成。其中汉字根据使用的频率分为两级。一级汉字 3755 个，二级汉字 3008 个。由于字符数量比较大，`GB2312` 采用了二维矩阵编码法对所有字符进行编码，所有的字符都可通过其区位码转换为数字编码信息。接下来，用 `pypinyin` 转成对应的拼音，加入集合。这里的数据结构使用了字典镶嵌列表。

小组成员尝试了 14 个常用词典库，如百度分词库、qq 拼音词库、搜狗词库等，最后选择了 `out.txt`，具体事例如图。

名称	类型	压缩大小	密码保护
30wChinsesSeqDic.txt	文本文档	3,210 KB	否
30wChinsesSeqDic_clean.txt	文本文档	1,743 KB	否
30wdict.txt	文本文档	1,476 KB	否
30wdict_utf8.txt	文本文档	1,735 KB	否
42537条伪原创词库.txt	文本文档	337 KB	否
dict.txt	文本文档	945 KB	否
fingerDic.txt	文本文档	244 KB	否
httpcws_dict.txt	文本文档	803 KB	否
out.txt	文本文档	761 KB	否
QQ拼音词库导出.txt	文本文档	1 KB	否
百度分词词库.txt	文本文档	436 KB	否
四十万汉语大词库.txt	文本文档	462 KB	否
四十万可用搜狗txt词库.txt	文本文档	382 KB	否
五笔词库.TXT	文本文档	181 KB	否

out.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
一个我们时间中可以公司没有信息软件下载软件注册自己品工作论坛企业这个他管理已经问题内容内进行市场服务如果系统技术发展现在者是就网络

out.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
首页方法希望地方也是单位怎么应该今天上新帖子显示能力电脑记者查看位置不由无法详细投资是一个进入发生这感觉更多你们的话起来标准

图 5-1 out.txt 数据集

将常用词典中的词过 pypinyin 转成拼音，加入集合。另外，为了适应网络语境，将一些如今网络常用缩写也加入集合，如“tql--太强了”、“szd--是真的”、“zqsg—真情实感”等。

5.2.2 拼音/单词识别

区分文本中的字母是英语单词还是汉语拼音，我们采用了二元语言模型（bigram model）。根据马尔可夫假设，任意一个词 w_i 的出现概率只与它前面的词 w_{i-1} 有关，则判断一句含有 n 个词语 $(w_1, w_2, \dots, w_n)$ 的句子是否合理，有公式：

$$P(w_1, w_2, \dots, w_n) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_2) \cdots P(w_n|w_{n-1})$$

映射到本模型中，判断一个含有 n 个字母 ($w_1, w_2, \dots, w_n$) 的词是否是英语单词，例如字母串 learn，计算 learn 可能是英语单词的概率 $P(\text{learn})$ ：

$$P(\text{learn}) = P(l | < BOS >) P(e | l) P(a | e) P(r | a) P(n | r) P(< EOS > | n)$$

其中，每一项条件概率的计算公式为：

$$P(w_i, w_{i-1}) = \frac{c(w_{i-1} w_i)}{\sum_{j=1}^n c(w_{j-1} w_j)}$$

具体实现方面，作为二元语音模型，首先实现一个计算单字母和双字母频数的函数，其中 symbol 是待计数的字母，corpus 是利用语料库所生产的单词字典：

```
def frequency(symbol, corpus):
    l = len(symbol)
    freq = 0
    for word in corpus.keys():
        freq_i = 0
        for i in range(len(word)):
            if l == 1:
                if word[i] == symbol:
                    freq_i += 1
            if l == 2:
                if word[i:i+2] == symbol:
                    # print(word)
                    freq_i += 1
        freq_i = freq_i * corpus[word]
        freq += freq_i
    return freq
```

条件概率计算实现：

```
def condition_prob(w1, w2, corpus):
    freq_w1 = frequency(w1, corpus)
    freq_w2 = frequency(w2, corpus)
    return (float(freq_w2)+1)/(float(freq_w1)+len(corpus.keys()))
```

主要公式实现：

```
def testing(word, corpus):
    cond_probs = []
    cond_p = condition_prob('>', '>' + word[0], corpus)
    cond_probs.append(cond_p)

    for i in range(len(word)-1):
        cond_p = condition_prob(word[i], word[i:i+2], corpus)
```



```
cond_probs.append(cond_p)

cond_p = condition_prob(word[-1],word[-1]+'<',corpus)
cond_probs.append(cond_p)

reliability = reduce(mul, cond_probs) * math.pow(10,len(word))
return reliability
```

5.2.3 拼音转汉字

①拼音分词

因为输入的拼音并不是提前分好词的，如“ni hao”，而是一串“nihao”这种类型的，所以需要先对拼音进行分词，这里采用了简单的正向最大匹配算法。

正向最大匹配算法（MM）的思想是假设自动分词中最长词条所含拼音的个数为 n ，则截取需要分词文本中当前字符串序列中的前 n 个字符作为匹配字段，查找分词词典，若词典中有这样一个 n 字词那么就匹配成功，匹配字段作为一个词被切分出来；若词典中找不到这样的 n 字词那么匹配失败，匹配字段去掉最后一个拼音，剩下的字符作为新的匹配字段再进行匹配，如此进行下去，直到匹配成功为止。具体步骤如下：

（1）导入分词词典 `pinyinLib.txt`，存储为字典形式 `dic`，需要分词的文本文件存储为字符串 `chars`

（2）遍历分词词典，找出最长的词，其长度为此算法中的最大分词长度 `max_chars`

（3）创建空列表 `words` 存储分词结果

（4）初始化字符串 `chars` 的分词起点 `n=0`

（5）判断分词点 n 是否在字符串 `chars` 内，即 $n < \text{len}(\text{chars})$ 如果成立，则进入下一步骤，否则进入（9）

（6）根据分词长度 i （初始值为 `max_chars`）截取相应的需分词文本 `chars` 的字符串 `s`

（7）判断 `s` 是否存在于分词词典中，若存在，那么直接添加到分词结果 `words` 中，分词起点 n 后移 i 位，转到步骤（5）

（8）将 `s` 添加到分词结果 `words` 中，分词起点 n 后移 1 位，转到步骤（5）

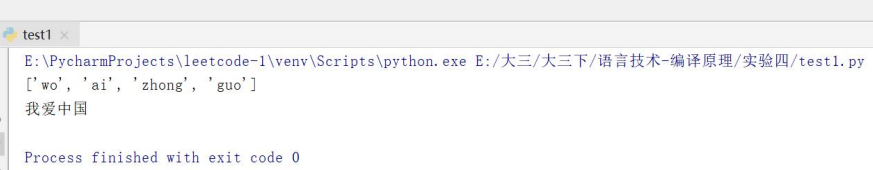
(9) 保存分词结果 words

②基于 HMM 将拼音转成汉字

转中文部分根据 HMM 实现了拼音转汉字的功能，这个部分在实验 2 中做过，这里为了方便，调用了 Pinyin2Hanzi 的 viterbi 函数进行实现。

③效果

```
35 pinyin_str=pinyin_or_word('~woaizhongguo~')
36 print(pinyin_str)
37 hmmparams = DefaultHmParams()
38 result = viterbi(hmm_params=hmmparams, observations=(pinyin_str), path_num = 1)
```



```
test1 x
E:\PycharmProjects\leetcode-1\venv\Scripts\python.exe E:/大三/大三下/语言技术-编译原理/实验四/test1.py
['wo', 'ai', 'zhong', 'guo']
我爱中国

Process finished with exit code 0
```

图 5-2 字母符号转换效果

5.3 拆字倒映射

这一步主要是要处理拆字型的不规范，例如“科比”会被拆成“禾斗匕匕”，我们的目的就是做到一个倒映射，即将“禾斗匕匕”映射为“科比”。当然，考虑到刻意拆字的情况并不多（我们也使用了常用词语词典，发现能够全部拆分的词不多），我们只做了对单个字拆字的倒映射。

这里主要使用 github 的开源项目 [kfcd/chaizi](#) 的拆字字典来实现。

我们的目的是做一个倒映射，即找到连续的几个汉字，需要用一个映射表来找到其对应的完整汉字。为了使映射表不至于过大，这里只对常见的汉字进行处理。我们认为，字符集 GB2312 中的字符就是常用字符，我们就依次对拆字字典进行了一次过滤。

因为在这个词典中，有些汉字拆分后只有一个汉字，实践证明，这样的情况会导致很多错误规范发生，于是我们又进行了第二遍的过滤，得到了较为干净的拆字倒映射表。

5.4 分类与选择策略

分类与选择策略可以说是本程序的核心策略。

考虑到对于每一种不规范的地方，我们都至少有两种选择：替换和不替换。所以对于每一处不规范的地方都是一个分类问题。

对于这种分类策略，我们选择 Word2Vec+上下文的策略。我们可以使用 Word2Vec 来获取一个词的词向量，并且认为，对于一处可能不规范的地方，几种替换词中和上下文最接近的词就是我们想要的结果。当然，这种想法也是基于以下的假设的：不规范的地方只出现在文本的少数几处，而其上下文都是规范文本。

于是，我们需要大量的语料来训练文本，并且，使用的是规范文本，这样，一些很明显的非规范词就会被很简单地被排除掉。这里使用 gensim 这个 python 库来做 Word2Vec 的训练。

对于分类与选择策略，我们只需要一个函数即可以解决。这个函数接受三个参数：Word2Vec 模型、候选结果列表和上下文列表。对于候选结果列表中每个候选结果，我们计算其与上下文列表所有词的相似度的加权平均值（目前暂时使用相似度的和），取值最大的一个候选结果（对于不在模型词典中的词，我们认为这个平均值为 0，事实上，这样的词很难成为真正的结果）。

这样，对于每一处可能的非规范的地方我们都采用这种做法，就可以得到一个比较好的替换效果。

5.5 匹配策略

综合以上 5.1-5.4 四个部分，我们将系统串连，搭建的系统主程序流程如下：

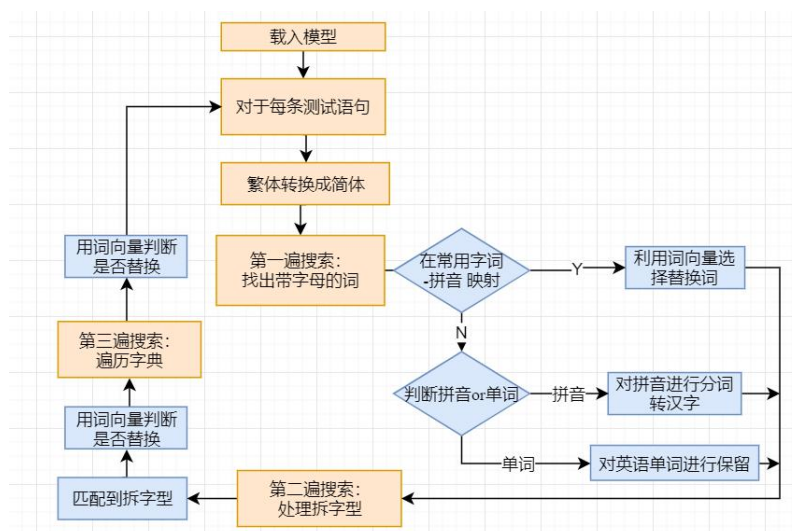


图 5-3 主程序流程图

1、载入各部分模型，主要有词向量模型、常用字词拼音映射、拆字型映射、英语二元语言模型。

对于每个测试语句进行如下处理：

2、对整个文本，先通过繁简转换逻辑，全部转为简体文本

3、进行第一遍搜索，找出带有字母的词

(1) 如果带有字母符号的词是常用字词的拼音映射，则用 `jieba` 分词工具将上下文进行分词，创建一个上下文词汇集合，然后将候选词和原词加入候选词集合，将上下文词汇集合和候选词集合一并通过词向量模型分类与选择策略，判断是否需要进行替换

(2) 如果带有字母符号的词不在常用字词的拼音映射，则判断是拼音还是英语单词，如果是拼音，则对拼音进行分词并转化成汉字，如果是英语单词，予以保留

4、第二遍搜索，匹配拆字型类

遍历拆字型列表，如果匹配到拆字型，用 `jieba` 分词工具将上下文进行分词，创建一个上下文词汇集合，然后将候选词和原词加入候选词集合，通过上下文词汇集合和候选词集合利用词向量判断是否需要替换

5、第三遍搜索，遍历字典

对于以上解决不了的问题，采用遍历字典的方式寻求解，如果匹配到字典项，同样还是通过词向量模型判断是否进一步替换

6、最后输出结果并计算正确率

5.6 命令行输入实现

为了使程序满足实验的要求，我们的代码需要以命令行的形式实现。

这里简单地使用 `sys` 库来进行命令行参数读取，并进行基本的输入参数判断（输入参数不足），如果输入的是文件，则默认文件是存在的，并且采用 `utf-8` 编码（可以在代码中调整）。

六、测试

6.1 测试结果

6.1.1 字符转换结果

如图所示：

```
'z' is a pinyin  
这可能是个英文单词！  
'qq' is a pinyin  
这可能是个英文单词！
```

图 6-1 字符转换结果-匹配部分

对于判断是英文还是拼音，我们通过双重检验，第一次检验是通过英语的二元语言模型，它倾向于判定字母符号为拼音，如果其认定是英语，那么它为英语的可能性极高，但是会漏判一些。

所以我们还有第二次检验，如果二元语言模型判断其是拼音，那我们将用正向最大匹配的方法进行二次校验，如果正向最大匹配没有办法将它匹配成拼音（因为它的结构不属于拼音声母-韵母结构），那我们将最终认定它为英语。

双重检验使我们的判定更加精准。

如图所示：

```
原句：虽然但是 看到有人说zhege世界怎么le  
转后：虽然但是 看到有人说这个世界怎么了  
正确：虽然但是 看到有人说这个世界怎么了
```

图 6-2 字符转换测试结果

对于“le”这个拼音，对应的汉字有很多，例如：“了、乐、勒”等，我们通过上下文，可以正确的选出“了”字，还是十分有效的。

6.1.2 拆字型转换结果

对于拆字型的问题，大部分处理效果还是比较理想的，例如：

```
原句：后来见了禾斗匕匕感动死了女马白勺！  
转后：后来见了科比感动死了妈的！  
正确：后来见了科比感动死了妈的！
```

图 6-3 拆字型识别效果 1

或者有：

```
原句：这和戴了套，不算弓虽女干一样有区别吗  
转后：这和戴了套，不算强奸一样有区别吗  
正确：这和戴了套，不算强奸一样有区别吗
```

图 6-4 拆字型识别效果 2

可以看出,对于上下文都是规范文本,只有少数地方是拆字型不规范的情况,我们很容易根据上下文找到正确的结果。

6.1.3 测试集识别结果

因为对于词准确率很难有自动的判别标准,所以我们选择句准确率作为测试指标,即一个句子经过转换后完全正确才认为转换正确。

对于测试集“规范化测试集 1000 条.txt”,一开始,我们有以下的识别结果(程序好像是多读入一行空行并进行识别,不过影响不大):



```
正确个数:
120
正确率:
0.12096774193548387
```

图 6-5 测试结果 1

于是,我们对程序进行了一些改造(整体思路没变,我们针对其中的一些问题做了更细致的判断),最终的测试结果为:



```
正确个数:
133
正确率:
0.13407258064516128
```

图 6-6 测试结果 2

虽然句准确率并不算特别高,但对于一个句子,部分规范化成功的例子还是很多的,考虑到这个问题本身就是有很大难度的,我们认为这个结果大体上是使我们满意的。

6.2 结果说明

测试结果说明,我们能够做到完全规范化的句子是不多的,但是查看测试结果发现,我们所考虑到的几种情况,效果还是可以的。

主要的一些难以识别的情况有:

(1) 错误叠加。例如,有些不规范文本是拆字后再进行同音替换,这时候,我们的程序就无能为力了。例如:

原句: 害貝妳終玗唻叻, 玕巳糸 至等妳ぬ久叻
转后: 害貝妳終玗唻叻, 玕巳糸 至等妳ぬ久叻
正确: 宝贝你终于来了, 我已经等你好久

图 6-7 错误叠加案例

(2) 整句不规范。因为我们的程序假定不规范的地方是出现在局部的，这样我们就可以通过上下文来决定应该对不规范的地方进行何种替换，可是当整句话都是不规范文本的话，这种策略就失效了，例如下面这种情况：

原句: 口斤讠兑这木羊能弓 | 起大家讠主意, 我来讠式讠式
转后: 口斤说这木羊能弓 | 起大家注意, 我来试试

图 6-8 整句不规范案例

如果要对这种情况进行处理的话，就需要增加某些整体的策略。

(3) 谐音问题。如图：

原句: 你是拿抓吗
转后: 你是拿抓吗
正确: 你是哪吒吗

图 6-9 谐音案例

(4) 词义分析类型。虽然我们是基于词向量模型的，但是因为对上下文的处理不够细致，导致这类问题的解决也不够理想，例如：

原句: 我的热评啊，急死我了
转后: 我的热评啊，急死我了
正确: 我的热度高的评论啊，急死我了

图 6-10 词义分析类型案例

(5) 半拼音半简写。如图：

原句: mad我又双叒叕来看了，之前上学有人用这首歌上台表演
'mad' is a english word
转后: mad我又又又又来看了，之前上学有人用这首歌上台不要演
正确: 妈的我又又又又来看了，之前上学有人用这首歌上台表演

图 6-11 半拼音半简写案例

对于这种半拼音半简写的词汇，暂时还无法解决，系统将其识别成为了英文。

(6) 其他问题。例如：

原句: 宿舍地位最低的我和我的高 (dou) 冷 (bi) 室友们
转后: 宿舍地位最低的我和我的高 (都) 冷 (必) 室友们
正确: 宿舍地位最低的我和我的逗逼室友们

图 6-12 其他问题案例

虽然可以将括号中的词转换过来，并且结合上下文判断是哪个字，但是并没有很好的办法将这两个字联系成一个词。

七、遇到的问题与解决

整体实现起来，我们面临的大问题比较少，主要有以下一些：

（1）训练集与测试集质量问题。这算是这类问题的一个通病，我们的训练集和测试集都不敢说是非常优质的，而使用这样的数据集做测试就会出现各种各样的问题，例如制作拆字倒映射的时候，就出现了一个字拆分成一个字的情况，例如“鸟”被拆分成“一”，导致匹配的时候有时会把“一”替换成“鸟”（特别是一处，上下文中有“崽”这个字）：



原句：刷名字弹幕挡视频的一群崽种麻烦再去做一遍弹幕礼仪考试题，顺便说一句今晚你马biss
转后：刷名字弹幕挡视频的鸟群崽种麻烦再去做鸟遍弹幕礼仪考试题，顺便说鸟句今晚你马必死
正确：刷名字弹幕挡视频的一群崽种麻烦再去做一遍弹幕礼仪考试题，顺便说一句今晚你妈必死

图 7-1 测试过程中识别错误一例

这导致了很很多替换错误的情况。后来，我们把倒映射表中这种情况给去掉了，准确率就提高了。

对于英语语料库也是这样，许多比较优秀的语料库都是收费的，所以最后用的语料库并不是很理想，导致“判断是拼音还是英语”这部分准确度略低。

虽然目前因为训练集和测试集比较小，可以做一些手动修改，但是当训练集和测试比较大时，就难以做这样的工作了。

（2）“判断是拼音还是英语”这部分最开始使用的最大正向匹配算法实现，效果不佳，后续采用了英语二元语言模型，但是会倾向于把“z”“qq”这种判断出拼音，最后使用的是英语二元语言模型+最大正向匹配算法双重检验，效果比较好。

（3）区分拼音单词后，单词不做处理，但下一步映射时可能把已经完成好的单词转换成其他错误形式，如将“China”转换为“Chin 啊”“C 很 a”。

八、感想

田佳铭：这次实验做起来还是挺花时间和精力，期末了，大家的实际都不太多，而且面对这个复杂的问题都不知道该怎么下手。最终，我们选择了一个比较简单的策略，聚集某一些错误，并且采用较为简单的方法来实现基于分类的规范化策略，使得整个思路比较具体，就可以展开工作了。

要考虑到所有的情况是很难的，经过我们的反复测试，句子正确率也只有13%左右，有些策略甚至会降低正确率，当然，这也是正常的现象。当然，我们发现在我们设想的较为理想的条件下的规划化效果还是很不错的，说明我们的策略在某些环境下还是有重要的作用的，这一点也使我们很欣慰。

数据集一直是一个头大的问题，训练集的好坏也会直接影响最终的效果。本次我们选择的一些训练集也不能说是很理想，训练的也不够充分，这些都限制了我们的准确率。总的来说，这个任务是兼具的，要想做出很好的规范化程序需要付出很大的努力。当然，在这个实验中，我们也学到了很多。

迟宇希：从一开始看到这个题目，就觉得是个很难的课题。变体词的规范问题是一个开放性问题，至今没有成熟的最优解决方案。

最初的几周我们查找了许多相关论文资料，但是论文中提到的方法都过于深奥，而且没有合适的数据集可供训练，而最简单的方法是使用词典替换，但是这样并不能覆盖大范围的问题，并且没有创新意义。我们经过讨论决定，对于每种不同的情况，运用不同的模块产生各自的候选集，最后通过词向量模型的方法对候选词进行选择。

我们选取了几种主要的错误情况，对于拼音、拆字、字母缩写等提出了各自的产生候选词的方法，对于错别字的情况本想用四角编码法，最终因为效果不佳等原因未能实现。

总体思路不难，但是到了具体实现的时候总会遇到各种各样的困难。

在我负责制作的常用字词的拼音映射部分，主要问题是选择一个合适的数据结构来存储映射，根据其一对多形式，和需要快速查找的特点，我最终选用了字典镶嵌列表的形式实现，另外一个难点是常用词的数据集问题，我找到了四十万

汉语大词库、百度分词词库、五笔词库等 14 个词库，并最终选择了合适的一个。

实现拼音转汉字时，因为 HMM 的实现我们之前做过，所以主要问题就是拼音的分词，最终我选择了较简单快速的正向匹配方法，虽然正向最大匹配会出现“open”可能被分词成“o, pen”的可能，但是因为我们前面加了一个“判断拼音还是英语单词”，所以“open”这类词会被首先过滤掉，并不会出现很多这种问题。

在编写主程序时，体会是自己的编程水平还是有待提高，有很多变量名的编写不够规范，在整合大家的代码时，会遇到许多冲突问题，函数也不够整齐。

我们的模型很多，但是最后经过慎重考虑，并没有合并成一个训练文件，因为这样不利于修改与扩充，如果合并成一个训练文件，更新任何一个模型都要重新训练所有模型，并不利于系统的调试。

本次实验用到了语言分析中的许多部分，例如分词、词向量、HMM、二元语言模型等，算是对所学知识的一次总结。

颜思琦：这次实验更像是本学期之前实验的综合应用，如简繁转换、马尔可夫模型、分词、word2ve 词向量模型、匹配算法等。本次实验需要考虑的问题很多，刚刚拿到题目时完全没有思路。后来在本组同学和组长的带领下一步步分析，将复杂的问题转化为多个部分，逐个寻找解决的办法，最后拼装整合，基本完成了实验要求。

通过这此实验，我在如何分析思考应用问题方面有了很大启发，观摩优秀的同学思考、解决问题的条理和逻辑，也让我学到了很多東西。我发现我在解构问题框架、查找相关资料、灵活应用新知识方面还有所欠缺，会在未来的学习中多加注意，勤于练习。